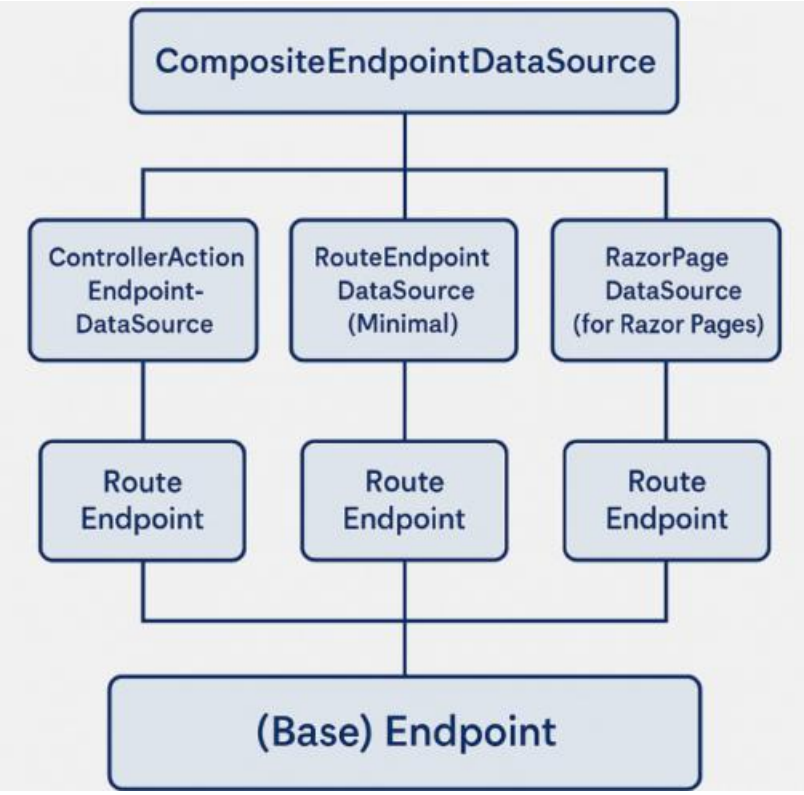
HOW ROUTING WORKS IN ASP.NET CORE WEB API

Understanding Core Routing Components

Routing is one of the most important internal mechanisms of ASP.NET Core. It dedicates which piece of code (controller action, Razor Page, Minimal API, etc.) handle a given HTTP request. To make this work efficiently, ASP.NET Core uses a few key internal classes:

- **Endpoint**
- **RouteEndpoint**
- **EndpointDataSource**
- **CompositeEndpointDataSource**

Together, they form the backbone of the Endpoint Routing System, the engine that connects incoming URLs to the correct request handlers. For a better understanding, please have a look at the following image, which shows the high-level overview of these classes and their relationships.



The Endpoint Class

At the heart of the Endpoint Routing System is the Endpoint class. Think of it as the universal contact that represents anything in ASP.NET Core capable of handling a request, whether that's a Controller Action, a Razor Page, a Minimal API, or even a SignalR Hub.

What an Endpoint contains:

- **RequestDelegate**: A compiled function that executes when this endpoint is matched.
- **Metadata Collection:** Attributes like [Authorize], [HttpGet], [Produces], or custom metadata used by middleware.
- **DisplayName:** Human-readable identifier used for debugging, logging, or displaying in tools like Swagger.

This abstraction allows ASP.NET Core to treat all kinds of request handlers uniformly.

The RouteEndpoint Class

While Endpoint is a general concept, many web applications depend on Route Templates. To represent such endpoints, ASP.NET Core provides a specialized subclass called RouteEndpoint.

What RouteEndpoint adds:

- **RoutePattern:** Presents the parsed structure of a Route Template (like api/products/{id?}, including Parameters, Constraints, and Default Values.
- **Order:** Defines the priority used to resolve conflict when multiple routes could match the same URL.

Why RouteEndpoint exists: No all endpoints follow a route template. SignalR hubs connect via a fixed hub path. Static file middleware doesn't use {controller}/{action} patterns. The RouteEndpoint is used only for endpoints that define route templates.

<u>The EndpointDataSource Class</u>

The **EndpointDataSource** acts as the central provider of all endpoints that the routing system can recognize. Each feature register its own dedicated EndpointDataSource during application startup.

Examples:

- MapControllers() → Registers **ControllerActionEndpointDataSource** (contains all controller actions)
- MapRazorPages() → Registers **PageActionEndpointDataSource** (contains all Razor Pages)
- MapGet() / MapPost() → Registers **RouteEndpointDataSource** (contains Minimal APIs)

The CompositeEndpointDataSource Class

Since multiple data source exists, the routing system needs a single unified source. The framework combines all individual data source into a unified collection called **CompositeEndpointDataSource**.

When the app starts:

- Each subsystem registers its own EndPointDataSource
- ASP.NET Core automatically creates a single CompositeEndpointDataSource that aggregates them
- The routing middleware queries this single composite when matching requests

How Routing Works – Step by Step

Application Startup Phase

Step 1: Application Startup Phase

When an ASP.NET Core application starts, the Program.cs file executes first.

Step 2: AddControllers() Registers MVC Service

When we call builder.Services.AddControllers(), ASP.NET Core registers all internal MVC services required for Web API functionality:

- Controller Discovery and Activation
- Model Binding and Validation
- Action and Result Filters
- Routing Metadata Providers

At this stage: No actual routes are created yet. Only services are registered.

Step 3: Build() Finalizes the Service Container

When we call app.Build(), ASP.NET Core finalizes the Dependency Injection container. Still, no routes exist in memory yet.

Step 4: MapControllers() Creates Endpoints

When app.MapControllers() executes:

- ASP.NET Core scans all assemblies for classes ending with "Controller"
- It reads [Route], [HttpGet], [HttpPost], etc. attributes
- It constructs **RouteEndpoint** objects for each action method

Each RouteEndpoint includes:

- Route Pattern (e.g., /api/books/{id})
- A RequestDelegate (compiled function that knows how to execute that controller action)
- A Metadata Collection (e.g., [Authorize], [HttpGet], custom filters)

These endpoints are then stored in memory inside the EndpointDataSource. From this point forward, routing never rebuilds these objects; they're created once at startup and reused every request.

Why no explicit app.UseRouting()?

Starting from .NET 6, app.MapControllers() automatically adds both Routing Middleware and Endpoint Middleware internally.

Request Processing Phase

Now, when a client sends a request: GET /api/books/5

Step 1: Request Enters the Middleware Pipeline

The request flows through middleware in the order they were registered.

Step 2: Routing Middleware Matches the Request

The Routing Middleware:

- Reads incoming request URL path (/api/books/5)
- Retrieves pre-built route patterns from EndpointDataSource
- Compares request path against patterns
- When a match is found:
  - Extracts route values (id = 5)
  - Selects the corresponding RouteEndpoint
  - Stores matched endpoint and route values in HttContext

At this stage: No controller logic has executed yet. The framework has only determined which action method will handle the request.

Step 3: Authorization Middleware Runs

If the matched endpoint has [Authorize] attribute, Authorization middleware checks user identity. If fails → returns 401 immediately.

Step 4: Endpoint Middleware Executes the Action

The Endpoint Middleware executes the associated **RequestDelegate**, which performs:

- **Model Binding –** Binds route values to action parameters
- **Validation –** Checks data annotations
- **Filter Execution –** Runs authorization, action, exception filters
- **Controller Activation –** Creates controller instance via DI
- **Action Invocation –** Calls the actual controller method
- **Result Execution –** Serializes response to JSON

Step 5: Response Sent Back to Client

The JSON response is written to HttpContext.Response and sent to client.

What is RequestDelegate?

RequestDelegate is a precompiled function that represents the entire process of executing a request for that endpoint. It is like a mini-pipeline containing all steps needed to execute the controller action. Each route has its own dedicated RequestDelegate.

When executed, it performs:

1) Model Binding → Maps route/query/body data to method parameters
2) Validation → Checks data annotations or validation rules
3) Filter Execution → Executes filters (authentication, exception, action, result)
4) Controller Activation → Creates controller instance via DI
5) Action Invocation → Calls the matched action method
6) Response Writing → Serialize result to JSON and writes to response

Now we are going to set up the project,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n LibraryApi6 -f net8.0

<u>Navigate to the project:</u>

Command: cd LibraryApi6

Install Packages

Before installing the packages, we should make sure that we are in the project folder 'LibraryApi6' in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

LibraryApi6.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Model

Path: LibraryApi6/Models/Book.cs

Book.cs

Models > Book.cs > ...

```
namespace LibraryApi6.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty;
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 2: Create In-Memory Data (10 Books)

Path: LibraryApi6/Data/BookData.cs

BookData.cs

Data > BookData.cs > ...

```
using LibraryApi6.Models;

namespace LibraryApi6.Data
{
    0 references
    public static class BookData
    {
        0 references
        public static List<Book> Books = new()
        {
            new Book {
                Id = 1,
                Title = "The Hobbit",
                Author = "J.R.R. Tolkien",
                Genre = "Fantasy",
                PublishedYear = 1937,
                IsAvailable = true,
                Price = 15.99m
            },
            new Book {
                Id = 2,
                Title = "1984",
                Author = "George Orwell",
                Genre = "Dystopian",
                PublishedYear = 1949,
                IsAvailable = false,
                Price = 12.99m
            },
            new Book {
                Id = 3,
                Title = "Pride and Prejudice",
                Author = "Jane Austen",
                Genre = "Romance",
                PublishedYear = 1813,
                IsAvailable = true,
                Price = 9.99m
            },
            new Book {
                Id = 4,
                Title = "The Great Gatsby",
                Author = "F. Scott Fitzgerald",
                Genre = "Classic",
                PublishedYear = 1925,
                IsAvailable = true,
                Price = 11.99m
            },
```

```
            new Book {
                Id = 5,
                Title = "Harry Potter",
                Author = "J.K. Rowling",
                Genre = "Fantasy",
                PublishedYear = 1997,
                IsAvailable = true,
                Price = 19.99m
            },
            new Book {
                Id = 6,
                Title = "Moby Dick",
                Author = "Herman Melville",
                Genre = "Adventure",
                PublishedYear = 1851,
                IsAvailable = false,
                Price = 14.99m
            },
```

```
            new Book {
                Id = 7,
                Title = "The Shining",
                Author = "Stephen King",
                Genre = "Horror",
                PublishedYear = 1977,
                IsAvailable = true,
                Price = 13.99m
            },
            new Book {
                Id = 8,
                Title = "Dune",
                Author = "Frank Herbert",
                Genre = "Sci-Fi",
                PublishedYear = 1965,
                IsAvailable = true,
                Price = 17.99m
            },
```

```
            new Book {
                Id = 9,
                Title = "The Alchemist",
                Author = "Paulo Coelho",
                Genre = "Fiction",
                PublishedYear = 1988,
                IsAvailable = true,
                Price = 10.99m
            },
            new Book {
                Id = 10,
                Title = "It Ends With Us",
                Author = "Colleen Hoover",
                Genre = "Romance",
                PublishedYear = 2016,
                IsAvailable = true,
                Price = 12.99m
            }
        };
    }
}
```

Step 3: Program.cs

Path: LibraryApi6/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Register MVC services (Controller Discovery, Model Binding, etc.)
// No routes created yet
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();      // Finalizes DI container

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();

// Creates RouteEndpoint objects and
// stores in CompositeEndpointDataSource
// NET 6+ automatically adds Routing Middleware + Endpoint Middleware
app.MapControllers();

app.Run();
```

Step 4: BooksController

Path: LibraryApi6/Controllers/BooksController.cs

BooksController.cs

Controllers > BooksController.cs > ...

```
using LibraryApi6.Data;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi6.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class BooksController : ControllerBase
    {
        // RouteEndpoint 1: GET /api/books
        [HttpGet]
        0 references
        public IActionResult GetAllBooks()
        {
            return Ok(BookData.Books);
        }

        // RouteEndpoint 2: GET /api/books/{id}
        [HttpGet("{id}")]
        0 references
        public IActionResult GetBookById(int id)
        {
            var book = BookData.Books
                .FirstOrDefault(b => b.Id == id);
            return
                book is not null ? Ok(book) : NotFound();
        }

        // RouteEndpoint 3: GET /api/books/genre/{genre}
        [HttpGet("genre/{genre}")]
        0 references
        public IActionResult GetByGenre(string genre)
        {
            var books = BookData.Books.Where(
                b => b.Genre.Equals(
                    genre, StringComparison.OrdinalIgnoreCase
                )
            ).ToList();
            return books.Any() ? Ok(books) : NotFound();
        }
    }
}
```

Step 5: DiagnosticsController (View Registered Endpoints)

Path: LibraryApi6/Controllers/DiagnosticsController.cs

DiagnosticsController.cs

Controllers > DiagnosticsController.cs > ...

```
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi6.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    1 reference
    public class DiagnosticsController : ControllerBase
    {
        // ASP.NET Core injects all registered
        // EndpointDataSource instances
        private readonly IEnumerable<EndpointDataSource>
            2 references
            _endpointDataSources;
        0 references
        public DiagnosticsController
        (IEnumerable<EndpointDataSource> endpointDataSources)
        {
            _endpointDataSources = endpointDataSources;
        }

        // GET: /api/diagnostics/endpoints
        // Returns all RouteEndpoint objects from
        // CompositeEndpointDataSurce
```

```
        // GET: /api/diagnostics/endpoints
        // Returns all RouteEndpoint objects from
        // CompositeEndpointDataSurce
        [HttpGet("endpoints")]
        0 references
        public IActionResult GetAllEndpoints()
        {
            var results = new List<object>();

            foreach (var dataSource in _endpointDataSources)
            {
                foreach (var endpoint in dataSource.Endpoints)
                {
                    results.Add(new
                    {
                        DisplayName = endpoint.DisplayName,
                        IsRouteEndpoint = endpoint is RouteEndpoint,
                        RoutePattern = (endpoint as RouteEndpoint)?
                            .RoutePattern?.RawText,
                        HttpMethods = endpoint.Metadata
                            .GetMetadata<HttpMethodMetadata>()?
                            .HttpMethods
                    });
                }
            }

            return Ok(results);
        }
    }
}
```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (LibraryApi6) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi6> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi6 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi6\bin\Debug\net8.0\LibraryApi
  6.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.48
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

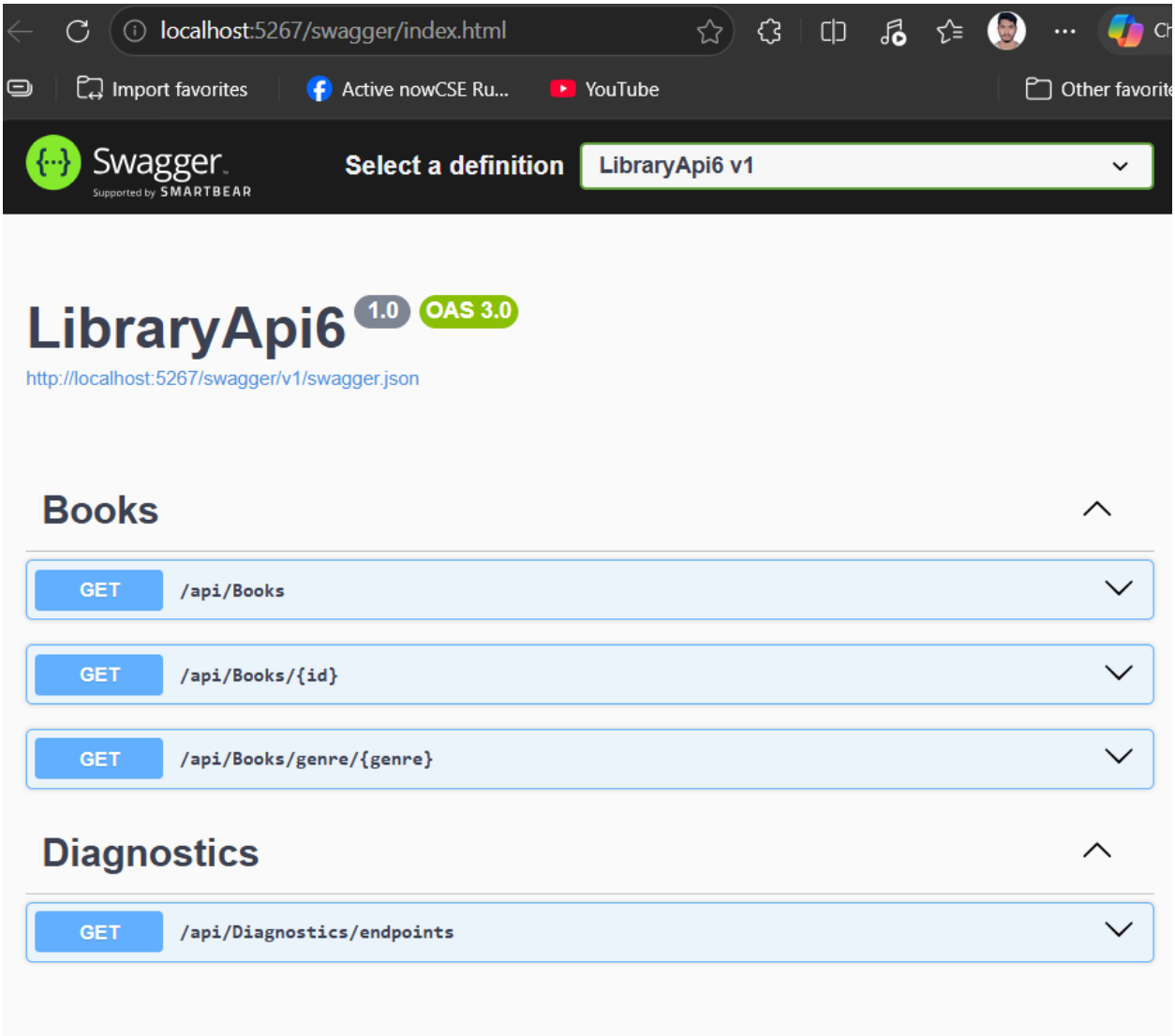
We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi6> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/ho
t-reload.
   💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi6 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi6\bin\Debug\net8.0\LibraryApi
  6.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5267
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi6
```

\


localhost:5267/swagger/index.html
Import favorites
Active nowCSE Ru...
YouTube
Other favorite
Swagger
Supported by SMARTBEAR
Select a definition
LibraryApi6 v1
LibraryApi6
1.0
OAS 3.0
http://localhost:5267/swagger/v1/swagger.json
Books
GET /api/Books
GET /api/Books/{id}
GET /api/Books/genre/{genre}
Diagnostics
GET /api/Diagnostics/endpoints

Test 1: View All Registered Endpoints

Request: GET http:localhost:5267/api/diagnostics/endpoints

Response (show RouteEndpoint objects in memory):

localhost:5267/api/diagnostics/endpoints

Import favorites | Active nowCSE Ru... | YouTube | Other favo

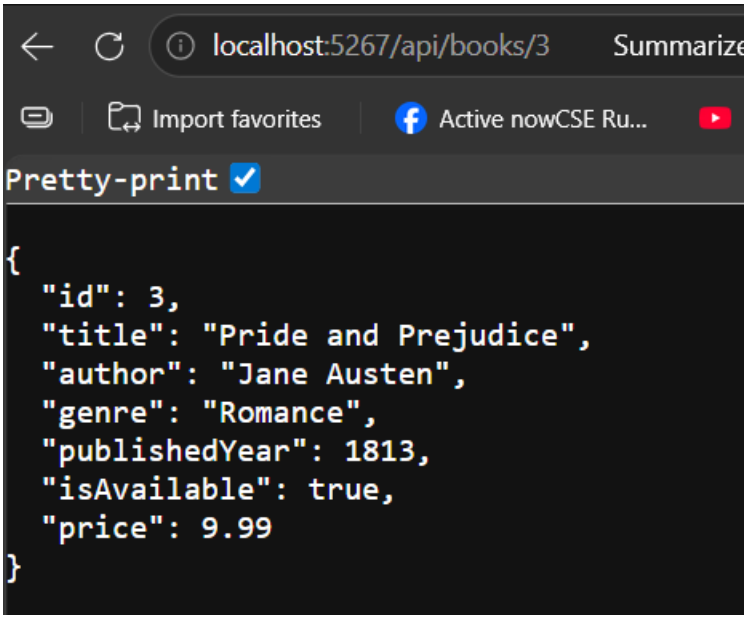
Pretty-print ☑


```
[
  {
    "displyName": "LibraryApi6.Controllers.BooksController.GetAllBooks (LibraryApi6)",
    "isRouteEndpoint": true,
    "routePattern": "api/Books",
    "httpMethods": [
      "GET"
    ]
  },
  {
    "displyName": "LibraryApi6.Controllers.BooksController.GetBookById (LibraryApi6)",
    "isRouteEndpoint": true,
    "routePattern": "api/Books/{id}",
    "httpMethods": [
      "GET"
    ]
  },
  {
    "displyName": "LibraryApi6.Controllers.BooksController.GetByGenre (LibraryApi6)",
    "isRouteEndpoint": true,
    "routePattern": "api/Books/genre/{genre}",
    "httpMethods": [
      "GET"
    ]
  },
  {
    "displyName": "LibraryApi6.Controllers.DiagnosticsController.GetAllEndpoints (LibraryApi6)",
    "isRouteEndpoint": true,
    "routePattern": "api/Diagnostics/endpoints",
    "httpMethods": [
      "GET"
    ]
  }
]
```

Test 2: Request Processed by Routing

Request: **GET http:localhost:5267/api/books/3**

Response:



Summary

Component	Purpose
Endpoint	Base classes (RequestDelegate + Metadata)
RouteEndpoint	Endpoint with RoutePattern
EndpointDataSource	Provides endpoints per subsystem
CompositeEndpointDataSource	Master collection for routing
MapControllers()	Creates RouteEndpoint objects at startup
Routing Middleware	Matches URL → attaches endpoint to HttpContext
Endpoint Middleware	Executes RequestDelegate